



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab Experiment – 01

8-Puzzle Problem

Aim

To solve the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm.

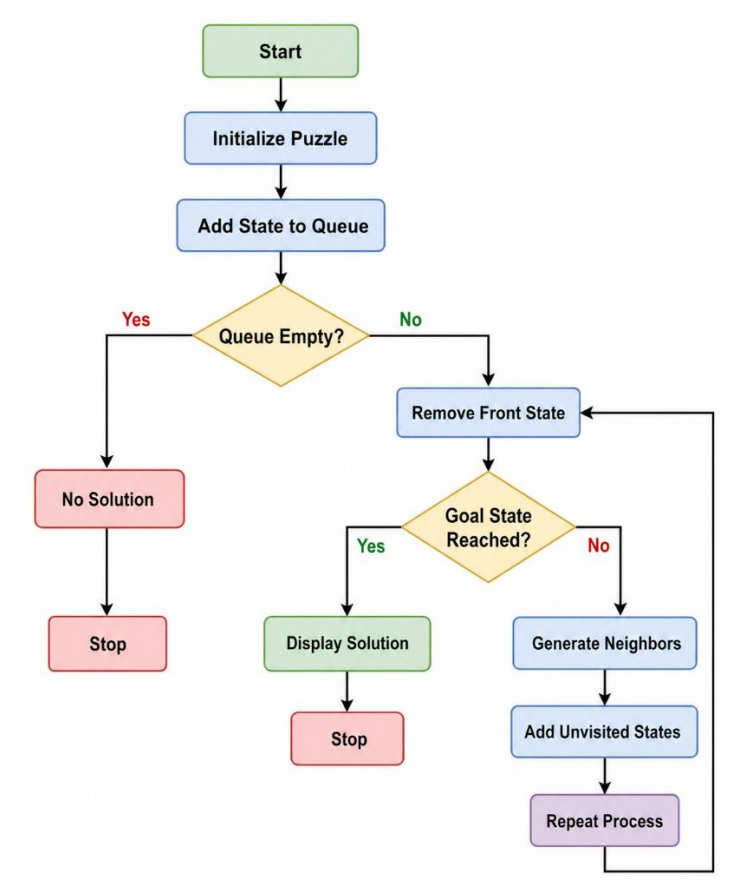
Objective

To find the shortest path from the initial state to the goal state by moving the blank tile.

Algorithm

1. Start with the initial puzzle state.
2. Insert the initial state into a queue.
3. Remove the first state from the queue.
4. Check whether it is the goal state.
5. If it is the goal state, display the solution.
6. Otherwise, generate all possible neighboring states.
7. Add all unvisited neighboring states to the queue.
8. Repeat the process until the goal state is reached or the queue becomes empty

FlowChart:



Code:

```
from collections import deque
```

```
goal = [1, 2, 3,  
        4, 5, 6,  
        7, 8, 0]
```

```
def get_neighbors(state):  
    neighbors = []  
    index = state.index(0)
```

```
moves = {  
    0: [1, 3],  
    1: [0, 2, 4],  
    2: [1, 5],  
    3: [0, 4, 6],  
    4: [1, 3, 5, 7],  
    5: [2, 4, 8],  
    6: [3, 7],  
    7: [4, 6, 8],  
    8: [5, 7]  
}
```

```
for move in moves[index]:  
    new_state = state[:]  
    new_state[index], new_state[move] = new_state[move],  
new_state[index]  
    neighbors.append(new_state)
```

```
return neighbors
```

```
def solve(start):  
    queue = deque([(start, [])])  
    visited = []
```

```
while queue:
```

```
    state, path = queue.popleft()
```

```
    if state == goal:
```

```
        return path + [state]
```

```
    if state in visited:
```

```
        continue
```

```
    visited.append(state)
```

```
    for neighbor in get_neighbors(state):
```

```
        queue.append((neighbor, path + [state]))
```

```
    return None
```

```
start = [1, 2, 3,
```

```
         4, 5, 6,
```

```
         0, 7, 8]
```

```
solution = solve(start)
```

```
if solution:
```

```
print("Solution Steps:")
```

```
for step in solution:
```

```
    print(step[0:3])
```

```
    print(step[3:6])
```

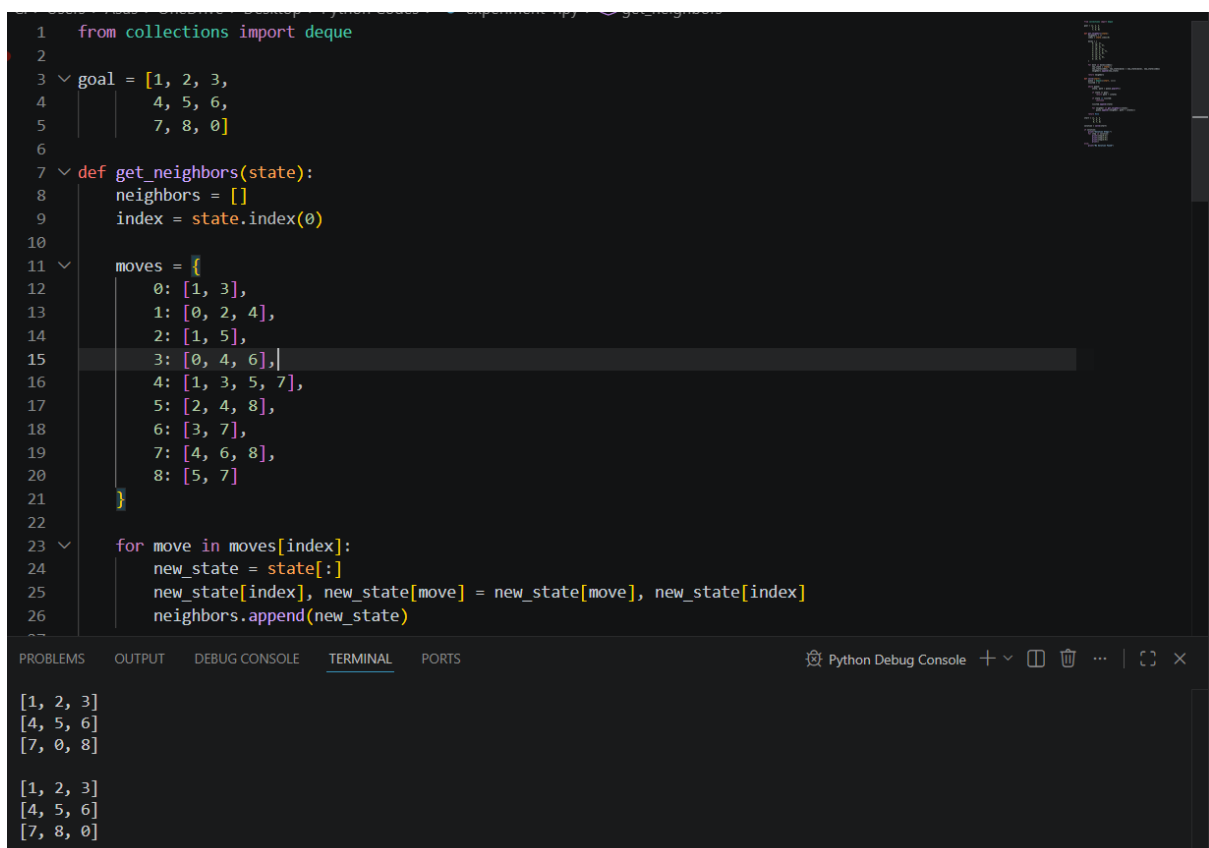
```
    print(step[6:9])
```

```
    print()
```

```
else:
```

```
    print("No Solution Found")
```

Output:



```
1  from collections import deque
2
3  goal = [1, 2, 3,
4          4, 5, 6,
5          7, 8, 0]
6
7  def get_neighbors(state):
8      neighbors = []
9      index = state.index(0)
10
11     moves = {
12         0: [1, 3],
13         1: [0, 2, 4],
14         2: [1, 5],
15         3: [0, 4, 6],
16         4: [1, 3, 5, 7],
17         5: [2, 4, 8],
18         6: [3, 7],
19         7: [4, 6, 8],
20         8: [5, 7]
21     }
22
23     for move in moves[index]:
24         new_state = state[:]
25         new_state[index], new_state[move] = new_state[move], new_state[index]
26         neighbors.append(new_state)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

Result

The 8-Puzzle problem is solved successfully using the Breadth-First Search (BFS) algorithm.

Conclusion

The BFS algorithm successfully finds the shortest sequence of moves required to reach the goal state from the initial state.